



Bash

This is a quick reference cheat sheet to getting started with linux bash shell scripting.

Getting Started

hello.sh

```
#!/bin/bash
```

```
VAR="world"
```

```
echo "Hello $VAR!" # => Hello world!
```

Execute the script

```
$ bash hello.sh
```

Variables

```
NAME="John"
```

```
echo ${NAME}      # => John (Variables)
```

```
echo $NAME        # => John (Variables)
```

```
echo "$NAME"      # => John (Variables)
```

```
echo '$NAME'      # => $NAME (Exact string)
```

```
echo "${NAME}!"   # => John! (Variables)
```

```
NAME = "John"     # => Error (about space)
```

Comments

```
# This is an inline Bash comment.
```

```
: '  
This is a  
very neat comment  
in bash  
'
```

Multi-line comments use `:` to open and `'` to close

Arguments

<code>\$1 ... \$9</code>	Parameter 1 ... 9
<code>\$0</code>	Name of the script itself
<code>\$1</code>	First argument
<code>\${10}</code>	Positional parameter 10
<code>\$#</code>	Number of arguments
<code>\$\$</code>	Process id of the shell
<code>\$*</code>	All arguments
<code>\$@</code>	All arguments, starting from first
<code>\$-</code>	Current options
<code>_</code>	Last argument of the previous command

See: [Special parameters](#)

Functions

```
get_name() {  
    echo "John"  
}  
  
echo "You are $(get_name)"
```

See: [Functions](#)

Conditionals

```
if [[ -z "$string" ]]; then  
    echo "String is empty"
```

```
elif [[ -n "$string" ]]; then
    echo "String is not empty"
fi
```

See: [Conditionals](#)

Brace expansion

```
echo {A,B}.js
```

```
{A,B}
```

Same as A B

```
{A,B}.js
```

Same as A.js B.js

```
{1..5}
```

Same as 1 2 3 4 5

See: [Brace expansion](#)

Shell execution

```
# => I'm in /path/of/current
```

```
echo "I'm in $(PWD)"
```

```
# Same as:
```

```
echo "I'm in `pwd`"
```

See: [Command substitution](#)

Bash Parameter expansions

Syntax

```
${F00%suffix}
```

Remove suffix

```
${F00#prefix}
```

Remove prefix

```
${F00%%suffix}
```

Remove long suffix

```
${F00##prefix}
```

Remove long prefix

```
${F00/from/to}
```

Replace first match

<code>\${F00//from/to}</code>	Replace all
<code>\${F00/%from/to}</code>	Replace suffix
<code>\${F00/#from/to}</code>	Replace prefix
Substrings	
<code>\${F00:0:3}</code>	Substring (position, length)
<code>\${F00:(-3):3}</code>	Substring from the right
Length	
<code>\${#F00}</code>	Length of <code>\$F00</code>
Default values	
<code>\${F00:-val}</code>	<code>\$F00</code> , or <code>val</code> if unset
<code>\${F00:=val}</code>	Set <code>\$F00</code> to <code>val</code> if unset
<code>\${F00:+val}</code>	<code>val</code> if <code>\$F00</code> is set
<code>\${F00:?message}</code>	Show message and exit if <code>\$F00</code> is unset

Substitution

```
echo ${food:-Cake}  #=> $food or "Cake"

STR="/path/to/foo.cpp"
echo ${STR%.cpp}    # /path/to/foo
echo ${STR%.cpp}.o  # /path/to/foo.o
echo ${STR%/*}      # /path/to

echo ${STR##*.}     # cpp (extension)
echo ${STR##*/}     # foo.cpp (basepath)

echo ${STR#*/}      # path/to/foo.cpp
echo ${STR##*/}     # foo.cpp

echo ${STR/foo/bar} # /path/to/bar.cpp
```

Slicing

```
name="John"
echo ${name}      # => John
echo ${name:0:2}  # => Jo
```

```
echo ${name::2}      # => Jo
echo ${name::-1}     # => Joh
echo ${name: (-1)}   # => n
echo ${name: (-2)}   # => hn
echo ${name: (-2):2} # => hn
```

```
length=2
```

```
echo ${name:0:length} # => Jo
```

See: [Parameter expansion](#)

basepath & dirpath

```
SRC="/path/to/foo.cpp"
```

```
BASEPATH=${SRC##*/}
```

```
echo $BASEPATH # => "foo.cpp"
```

```
DIRPATH=${SRC%$BASEPATH}
```

```
echo $DIRPATH # => "/path/to/"
```

Transform

```
STR="HELLO WORLD!"
```

```
echo ${STR,} # => hELLO WORLD!
```

```
echo ${STR,,} # => hello world!
```

```
STR="hello world!"
```

```
echo ${STR^} # => Hello world!
```

```
echo ${STR^^} # => HELLO WORLD!
```

```
ARR=(hello World)
```

```
echo "${ARR[@],}" # => hello world
```

```
echo "${ARR[@]^}" # => Hello World
```

Bash Arrays

Defining arrays

```
Fruits=('Apple' 'Banana' 'Orange')
```

```
Fruits[0]="Apple"
```

```
Fruits[1]="Banana"
```

```
Fruits[2]="Orange"
```

```
ARRAY1=(foo{1..2}) # => foo1 foo2
```

```
ARRAY2=({A..D})    # => A B C D
```

```
# Merge => foo1 foo2 A B C D
```

```
ARRAY3=(${ARRAY1[@]} ${ARRAY2[@]})
```

```
# declare construct
```

```
declare -a Numbers=(1 2 3)
```

```
Numbers+=(4 5) # Append => 1 2 3 4 5
```

Indexing

<code>\${Fruits[0]}</code>	First element
<code>\${Fruits[-1]}</code>	Last element
<code>\${Fruits[*]}</code>	All elements
<code>\${Fruits[@]}</code>	All elements
<code>\${#Fruits[@]}</code>	Number of all
<code>\${#Fruits}</code>	Length of 1st
<code>\${#Fruits[3]}</code>	Length of nth
<code>\${Fruits[@]:3:2}</code>	Range
<code>\${!Fruits[@]}</code>	Keys of all

Iteration

```
Fruits=('Apple' 'Banana' 'Orange')
```

```
for e in "${Fruits[@]}"; do  
    echo $e  
done
```

With index

```
for i in "${!Fruits[@]}"; do
```

```
printf "%s\t%s\n" "$i" "${Fruits[$i]}"
done
```

Operations

```
Fruits=("${Fruits[@]}" "Watermelon")    # Push
Fruits+=('Watermelon')                  # Also Push
Fruits=( ${Fruits[@]/Ap*/} )            # Remove by regex match
unset Fruits[2]                          # Remove one item
Fruits=("${Fruits[@]}")                  # Duplicate
Fruits=("${Fruits[@]}" "${Veggies[@]}") # Concatenate
lines=(`cat "logfile"`)                  # Read from file
```

Arrays as arguments

```
function extract()
{
    local -n myarray=$1
    local idx=$2
    echo "${myarray[$idx]}"
}
Fruits=('Apple' 'Banana' 'Orange')
extract Fruits 2    # => Orange
```

Bash Dictionaries

Defining

```
declare -A sounds
```

```
sounds[dog]="bark"
sounds[cow]="moo"
sounds[bird]="tweet"
sounds[wolf]="howl"
```

Working with dictionaries

```
echo ${sounds[dog]} # Dog's sound
echo ${sounds[@]}   # All values
```

```
echo ${!sounds[@]} # All keys
echo ${#sounds[@]} # Number of elements
unset sounds[dog]  # Delete dog
```

Iteration

```
for val in "${sounds[@]}; do
    echo $val
done
```

```
for key in "${!sounds[@]}; do
    echo $key
done
```

Bash Conditionals

Integer conditions

<code>[[NUM -eq NUM]]</code>	Equal
<code>[[NUM -ne NUM]]</code>	Not equal
<code>[[NUM -lt NUM]]</code>	Less than
<code>[[NUM -le NUM]]</code>	Less than or equal
<code>[[NUM -gt NUM]]</code>	Greater than
<code>[[NUM -ge NUM]]</code>	Greater than or equal
<code>((NUM < NUM))</code>	Less than
<code>((NUM <= NUM))</code>	Less than or equal
<code>((NUM > NUM))</code>	Greater than
<code>((NUM >= NUM))</code>	Greater than or equal

String conditions

<code>[[-z STR]]</code>	Empty string
---------------------------	--------------

<code>[[-n STR]]</code>	Not empty string
<code>[[STR == STR]]</code>	Equal
<code>[[STR = STR]]</code>	Equal (Same above)
<code>[[STR < STR]]</code>	Less than (ASCII)
<code>[[STR > STR]]</code>	Greater than (ASCII)
<code>[[STR != STR]]</code>	Not Equal
<code>[[STR =~ STR]]</code>	Regexp

Example

String

```
if [[ -z "$string" ]]; then
    echo "String is empty"
elif [[ -n "$string" ]]; then
    echo "String is not empty"
else
    echo "This never happens"
fi
```

Combinations

```
if [[ X && Y ]]; then
    ...
fi
```

Equal

```
if [[ "$A" == "$B" ]]; then
    ...
fi
```

Regex

```
if [[ '1. abc' =~ ([a-z]+) ]]; then
    echo ${BASH_REMATCH[1]}
fi
```

Smaller

```
if (( $a < $b )); then
    echo "$a is smaller than $b"
```

```
fi
```

Exists

```
if [[ -e "file.txt" ]]; then
    echo "file exists"
fi
```

File conditions

<code>[[-e FILE]]</code>	Exists
<code>[[-d FILE]]</code>	Directory
<code>[[-f FILE]]</code>	File
<code>[[-h FILE]]</code>	Symlink
<code>[[-s FILE]]</code>	Size is > 0 bytes
<code>[[-r FILE]]</code>	Readable
<code>[[-w FILE]]</code>	Writable
<code>[[-x FILE]]</code>	Executable
<code>[[f1 -nt f2]]</code>	f1 newer than f2
<code>[[f1 -ot f2]]</code>	f2 older than f1
<code>[[f1 -ef f2]]</code>	Same files

More conditions

<code>[[-o noclobber]]</code>	If OPTION is enabled
<code>[[! EXPR]]</code>	Not
<code>[[X && Y]]</code>	And
<code>[[X Y]]</code>	Or

logical and, or

```
if [ "$1" = 'y' -a $2 -gt 0 ]; then
    echo "yes"
fi
```

```
if [ "$1" = 'n' -o $2 -lt 0 ]; then
    echo "no"
fi
```

Bash Loops

Basic for loop

```
for i in /etc/rc.*; do
    echo $i
done
```

C-like for loop

```
for ((i = 0 ; i < 100 ; i++)); do
    echo $i
done
```

Ranges

```
for i in {1..5}; do
    echo "Welcome $i"
done
```

With step size

```
for i in {5..50..5}; do
    echo "Welcome $i"
done
```

Auto increment

```
i=1
while [[ $i -lt 4 ]]; do
    echo "Number: $i"
    ((i++))
done
```

Auto decrement

```
i=3
while [[ $i -gt 0 ]]; do
    echo "Number: $i"
    ((i--))
done
```

Continue

```
for number in $(seq 1 3); do
    if [[ $number == 2 ]]; then
        continue;
    fi
    echo "$number"
done
```

Break

```
for number in $(seq 1 3); do
    if [[ $number == 2 ]]; then
        # Skip entire rest of loop.
        break;
    fi
    # This will only print 1
    echo "$number"
done
```

Until

```
count=0
until [ $count -gt 10 ]; do
    echo "$count"
    ((count++))
done
```

Forever

```
while true; do
    # here is some code.
done
```

Forever (shorthand)

```
while ;; do
    # here is some code.
```

done

Reading lines

```
cat file.txt | while read line; do
    echo $line
done
```

Bash Functions

Defining functions

```
myfunc() {
    echo "hello $1"
}

# Same as above (alternate syntax)
function myfunc() {
    echo "hello $1"
}

myfunc "John"
```

Returning values

```
myfunc() {
    local myresult='some value'
    echo $myresult
}

result="$(myfunc)"
```

Raising errors

```
myfunc() {
    return 1
}

if myfunc; then
```

```
    echo "success"
else
    echo "failure"
fi
```

Bash Options

Options

```
# Avoid overlay files
# (echo "hi" > foo)
set -o noclobber

# Used to exit upon error
# avoiding cascading errors
set -o errexit

# Unveils hidden failures
set -o pipefail

# Exposes unset variables
set -o nounset
```

Glob options

```
# Non-matching globs are removed
# ('*.foo' => '')
shopt -s nullglob

# Non-matching globs throw errors
shopt -s failglob

# Case insensitive globs
shopt -s nocaseglob

# Wildcards match dotfiles
# ("*.sh" => ".foo.sh")
shopt -s dotglob

# Allow ** for recursive matches
# ('lib/**/*.rb' => 'lib/a/b/c.rb')
```

```
shopt -s globstar
```

Bash History

Commands

<code>history</code>	Show history
<code>sudo !!</code>	Run the previous command with sudo
<code>shopt -s histverify</code>	Don't execute expanded result immediately

Expansions

<code>!\$</code>	Expand last parameter of most recent command
<code>!*</code>	Expand all parameters of most recent command
<code>!-n</code>	Expand <code>n</code> th most recent command
<code>!n</code>	Expand <code>n</code> th command in history
<code>!<command></code>	Expand most recent invocation of command <code><command></code>

Operations

<code>!!</code>	Execute last command again
<code>!!:s/<FROM>/<TO>/</code>	Replace first occurrence of <code><FROM></code> to <code><TO></code> in most recent command
<code>!!:gs/<FROM>/<TO>/</code>	Replace all occurrences of <code><FROM></code> to <code><TO></code> in most recent command
<code>!\$:t</code>	Expand only basename from last parameter of most recent command
<code>!\$:h</code>	Expand only directory from last parameter of most recent command
<code>!!</code> and <code>!\$</code> can be replaced with any valid expansion.	

Slices

<code>!!:n</code>	Expand only <code>n</code> th token from most recent command (command is <code>0</code> ; first argument is <code>1</code>)
<code>!^</code>	Expand first argument from most recent command

!\$

Expand last token from most recent command

!!:n-m

Expand range of tokens from most recent command

!!:n-\$

Expand `n`th token to last from most recent command

!! can be replaced with any valid expansion i.e. `!cat`, `!-2`, `!42`, etc.

Miscellaneous

Numeric calculations

```
$(a + 200)    # Add 200 to $a
```

```
$(($RANDOM%200)) # Random number 0..199
```

Subshells

```
(cd somedir; echo "I'm now in $PWD")  
pwd # still in first directory
```

Inspecting commands

```
command -V cd  
#=> "cd is a function/alias/whatever"
```

Redirection

```
python hello.py > output.txt    # stdout to (file)  
python hello.py >> output.txt   # stdout to (file), append  
python hello.py 2> error.log    # stderr to (file)  
python hello.py 2>&1            # stderr to stdout  
python hello.py 2>/dev/null     # stderr to (null)  
python hello.py &>/dev/null     # stdout and stderr to (null)  
  
python hello.py < foo.txt       # feed foo.txt to stdin for python
```

Source relative

```
source "${0%/*}/../share/foo.sh"
```


Directory of script

```
DIR="${0%/*}"
```

Case/switch

```
case "$1" in
    start | up)
        vagrant up
        ;;

    *)
        echo "Usage: $0 {start|stop|ssh}"
        ;;
esac
```

Trap errors

```
trap 'echo Error at about $LINENO' ERR
```

or

```
traperr() {
    echo "ERROR: ${BASH_SOURCE[1]} at about ${BASH_LINENO[0]}"
}

set -o erretrace
trap traperr ERR
```

printf

```
printf "Hello %s, I'm %s" Sven Olga
#=> "Hello Sven, I'm Olga

printf "1 + 1 = %d" 2
#=> "1 + 1 = 2"

printf "Print a float: %f" 2
#=> "Print a float: 2.000000"
```

Getting options

```
while [[ "$1" =~ ^- && ! "$1" == "--" ]]; do case $1 in
```

```

-V | --version )
echo $version
exit
;;
-s | --string )
shift; string=$1
;;
-f | --flag )
flag=1
;;
esac; shift; done
if [[ "$1" == '--' ]]; then shift; fi

```

Check for command's result

```

if ping -c 1 google.com; then
    echo "It appears you have a working internet connection"
fi

```

Special variables

\$?	Exit status of last task
\$!	PID of last background task
\$\$	PID of shell
\$0	Filename of the shell script

See [Special parameters](#).

Grep check

```

if grep -q 'foo' ~/.bash_history; then
    echo "You appear to have typed 'foo' in the past"
fi

```

Backslash escapes

	!	"	#
&	'	(	)
,	;	<	>
[		\	]

^

{

}

`

\$

*

?

Escape these special characters with \

Heredoc

```
cat <<END
hello world
END
```

Go to previous directory

```
pwd # /home/user/foo
cd bar/
pwd # /home/user/foo/bar
cd -
pwd # /home/user/foo
```

Reading input

```
echo -n "Proceed? [y/n]: "
read ans
echo $ans

read -n 1 ans    # Just one character
```

Conditional execution

```
git commit && git push
git commit || echo "Commit failed"
```

Strict mode

```
set -euo pipefail
IFS=$'\n\t'
```

See: [Unofficial bash strict mode](#)

Optional arguments

```
args=("$@")
```

```
args+=(foo)
args+=(bar)
echo "${args[@]}"
```

Put the arguments into an array and then append

Also see

[Devhints](#) (devhints.io)

[Bash-hackers wiki](#) (bash-hackers.org)

[Shell vars](#) (bash-hackers.org)

[Learn bash in y minutes](#) (learnxinyminutes.com)

[Bash Guide](#) (mywiki.woledge.org)

[ShellCheck](#) (shellcheck.net)

[shell - Standard Shell](#) (devmanual.gentoo.org)

Related Cheatsheet

Awk Cheatsheet

Quick Reference

Python Cheatsheet

Quick Reference

Recent Cheatsheet

Remote Work Revolution Cheatsheet

Quick Reference

Homebrew Cheatsheet

Quick Reference

PyTorch Cheatsheet

Quick Reference

Taskset Cheatsheet

Quick Reference